МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»
Факультет: компьютерных систем и сетей
Кафедра: электронных вычислительных машин
Дисциплина: Жизненный цикл разработки программного обеспечения

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 3

Тема: «Разработка и документирование бизнес-требований»

Студент: Н.А.Шурхай
Преподаватель: О.В.Внук

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Целью настоящей лабораторной работы является изучение процесса выявления требований к программному обеспечению, приобретение навыков их документирования в виде спецификации требований (SRS) по отраслевому шаблону, а также получение опыта совместной работы с требованиями в формате взаимодействия «Заказчик – Менеджер проекта» с использованием системы контроля версий Git и механизма pull request платформы GitHub.

Для достижения цели необходимо решить следующие задачи:

– выбрать проект на весь курс: определить вариант приложения, тип проекта, внешние сервисы, технологический стек, границы системы и название продукта;

– выявить требования: провести интервью с Заказчиком в рамках ролевой игры, сформулировать бизнес-цели, классы пользователей и границы продукта;

– разработать графическую часть документа на языке SysML v2 (контекстную модель, варианты использования и прототипы пользовательского интерфейса) и сохранить её в формате PNG;

– заполнить шаблон srs/srs-template.md, присвоив требованиям уникальные идентификаторы и критерии приёмки;

– зафиксировать документ в отдельной ветке, получить замечания бизнес-сообщества через pull request, проанализировать их (принять конструктивные и обоснованно отклонить остальные) и внести корректировки.

2 ЗАДАНИЕ И ИСХОДНЫЕ ДАННЫЕ

2.1 Условие задания

Первая часть задания выполняется в паре в виде ролевой игры: один студент играет роль Заказчика, второй – роль Менеджера проекта, задача которого – собрать набор требований к разрабатываемому программному обеспечению. Затем студенты меняются ролями и повторяют то же самое для другой темы. Начиная с данной лабораторной работы роль Заказчика играет напарник, назначенный преподавателем, а затем сам преподаватель, который может попросить внести корректировки на любом этапе разработки.

В соответствии с заданием необходимо:

– выбрать вариант приложения и тип проекта (Web-, Desktop- или Mobile-приложение), внешние сервисы, предпочтительные технологии, границы, язык описания и язык разработки, придумать название проекта и сообщить его преподавателю;

– получить напарника от преподавателя и бизнес-требования от Заказчика;

– в репозитории, созданном в лабораторной работе № 2, разработать графическую часть документа на языке SysML v2 (например, мокапы) и сохранить полученный результат в формате PNG;

– заполнить шаблон документа srs/srs-template.md и добавить в него ссылки на созданные PNG-файлы;

– сделать коммит в отдельную ветку;

– получить пожелания бизнес-сообщества (от напарника, затем от преподавателя) в виде pull request;

– проанализировать замечания и предложения, выбрать только конструктивные, обосновать отклонение остальных и внести корректировки по результатам взаимодействия с напарником;

– поддерживать репозиторий в актуальном состоянии, до окончательного оформления документов вести их разработку в отдельной ветке;

– ответить на контрольные вопросы и оформить отчёт согласно действующему СТП БГУИР.

2.2 Исходные данные

Для выполнения лабораторной работы были установлены следующие исходные условия:

– вариант задания – № 1, приложение для предоставления информации о курсах валют; выбранное направление – архивные курсы для расчёта налогообложения индивидуального предпринимателя;

– название проекта – «КурсНалог» (RateTax);

– тип проекта – desktop-приложение для Windows 10/11 на языке C++17 с графическим интерфейсом на классическом WinAPI (графики – GDI,

сетевое взаимодействие – WinINet, локальное хранилище – текстовые файлы в каталоге %LOCALAPPDATA%\RateTax) без сторонних фреймворков и библиотек;

– внешний сервис – открытый REST API Национального банка Республики Беларусь (api.nbrb.by), не требующий регистрации и ключа доступа; выбор обоснован тем, что для налогового учёта в Республике Беларусь применяется именно официальный курс НБ РБ;

– язык документирования – русский, формат – Markdown, графическая часть – SysML v2;

– репозиторий – личный форк akutkola-source/SDLC, созданный в лабораторной работе № 2; шаблон документа – srs/srs-template.md; материалы работы размещаются в отдельной папке lab3, рабочая ветка – lab3;

– роли – в первой части работы автор выступает Менеджером проекта, а напарник, назначенный преподавателем, – Заказчиком, затем роли меняются для проекта напарника.

3 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАБОТЫ С ТРЕБОВАНИЯМИ

3.1 Понятие требования и уровни требований

Требование к программному обеспечению – это условие или возможность, которой должна обладать система, чтобы удовлетворить договор, стандарт, спецификацию или иной формально установленный документ, а также потребность пользователя в решении задачи или достижении цели. В инженерии требований принято выделять несколько уровней:

– бизнес-требования – цели организации или Заказчика, ради которых создаётся продукт (например, «сократить время подготовки декларации с трёх часов до 15 минут»);

– пользовательские требования – задачи, которые пользователи должны решать с помощью системы, как правило, оформляемые в виде вариантов использования (use case) или пользовательских историй;

– функциональные требования – поведение системы, которое должно быть реализовано, чтобы пользователи могли выполнить свои задачи;

– нефункциональные требования (атрибуты качества) – производительность, безопасность, надёжность, удобство использования и т. д;

– ограничения – условия, сужающие пространство проектных решений (платформа, технологии, сроки, бюджет).

3.2 Процесс выявления требований

Выявление требований (elicitation) – итеративный процесс сбора информации у заинтересованных лиц. Основные методы: интервью, анкетирование, наблюдение за работой пользователя, анализ существующих документов и систем, мозговой штурм, прототипирование. В данной работе использовались интервью (ролевая игра) и прототипирование интерфейса: мокапы позволяют Заказчику увидеть будущую систему до начала разработки и уточнить ожидания.

Важно, что Заказчик в начале проекта, как правило, формулирует проблему, а не решение. Задача Менеджера – выяснить истинные бизнес-цели, определить границы продукта (что входит и что не входит в объём) и перевести пожелания в проверяемые требования.

3.3 Спецификация требований к программному обеспечению

Спецификация требований к ПО (Software Requirements Specification, SRS) – документ, который фиксирует, что должна делать система, но не как она будет реализована. Структура используемого шаблона соответствует IEEE 830 и ISO/IEC/IEEE 29148. Качественные требования должны быть:

– корректными и необходимыми, т. е. отражать реальную потребность;

– однозначными, т. е. допускать единственное толкование;

– проверяемыми, т. е. иметь критерий приёмки и метод верификации (Test, Analysis, Inspection, Demonstration);

– трассируемыми, т. е. иметь уникальный идентификатор вида REQ-[AREA]-[NNN];

– непротиворечивыми и осуществимыми в рамках ограничений проекта.

3.4 Язык моделирования SysML v2

SysML v2 – язык системного моделирования, стандартизированный OMG. В отличие от первой версии, основанной на профиле UML, SysML v2 имеет собственное метаядро KerML и равноправные текстовую и графическую нотации. Текстовая модель хранится в обычных файлах .sysml, что позволяет версионировать её в Git наравне с документацией. Основные конструкции, использованные в работе:

– part def / part – определение и использование структурного элемента (актор, система, экран, элемент интерфейса);

– attribute – свойство элемента (заголовок окна, подпись кнопки, координаты);

– port def / item def – точки взаимодействия и передаваемые данные (запрос курса, отчёт);

– use case def – вариант использования с указанием subject и actor, отношение include;

– state def – состояния и переходы (навигация между экранами).

4 ХОД ВЫПОЛНЕНИЯ РАБОТЫ

4.1 Выбор варианта и определение проекта

Из предложенных вариантов был выбран вариант № 1 в подварианте «архивные курсы для расчёта налогообложения индивидуального предпринимателя». Предметная область понятна и имеет реальную потребность: ИП, получающий выручку в иностранной валюте, обязан пересчитывать её в белорусские рубли по официальному курсу НБ РБ на дату поступления. Проект получил название «КурсНалог» (RateTax). В качестве языка разработки выбран C++, графический интерфейс предполагается строить средствами классического WinAPI, освоенного в лабораторной работе № 1. Такой подход не требует сторонних фреймворков: все используемые системные библиотеки (user32, gdi32, comctl32, wininet) входят в состав Windows. Тип проекта, стек технологий и границы были сообщены преподавателю и согласованы.

4.2 Ролевая игра: интервью с Заказчиком

Напарник в роли Заказчика сформулировал проблему: каждый квартал приходится вручную искать курс на сайте НБ РБ для каждого платежа и пересчитывать суммы в электронной таблице, что занимает много времени и приводит к ошибкам. Основные вопросы и ответы интервью приведены в таблице 4.1.

Таблица 4.1 – Протокол интервью с Заказчиком

Вопрос Менеджера проекта	Ответ Заказчика
Кто будет пользоваться приложением?	Я как ИП-фрилансер; иногда мой бухгалтер.
Какую проблему решаем?	Ручной поиск курсов и пересчёт в Excel — долго, бывают ошибки.
Какие валюты нужны?	USD, EUR, RUB, иногда CNY и PLN; пусть их можно выбирать.
Нужен ли расчёт налога?	Да, по ставке, которую я укажу сам.
Что должно получиться на выходе?	Таблица для бухгалтера и PDF-документ для себя.
Нужна ли постоянная связь с Интернетом?	Желательно, чтобы приложение работало и без неё.

По результатам интервью были сформулированы бизнес-цели продукта:

– БЦ-1 – сократить время ежеквартальной подготовки данных для налоговой декларации примерно с трёх часов до 15 минут;

– БЦ-2 – исключить ошибки пересчёта из-за неверной даты курса или неучтённого масштаба (например, курс за 100 RUB);

– БЦ-3 – обеспечить подтверждаемый источник курсов (официальные данные НБ РБ).

Также были определены границы: в объём входят получение курсов, книга учёта доходов, расчёт налога по заданной ставке, график динамики и экспорт отчётов; в объём не входят подача декларации, учёт расходов, банковские операции, мобильная и web-версии.

4.3 Подготовка репозитория и создание рабочей ветки

Перед началом работы локальный репозиторий был синхронизирован с удалённым. При этом было обнаружено, что удалённая ветка origin/main опережает локальную на два коммита (файлы лабораторной работы № 1 были загружены через веб-интерфейс GitHub). Чтобы pull request не содержал расхождений, основная ветка была обновлена методом Fast-Forward, и уже от её актуальной вершины была создана отдельная ветка lab3. Все материалы лабораторной работы размещаются в отдельной папке lab3 в корне репозитория – по аналогии с папкой lab1 первой лабораторной работы. Выполненные команды приведены в листинге А.1 приложения А, итоговая структура папки lab3 – в листинге А.2.

4.4 Разработка графической части на языке SysML v2

Графическая часть документа была разработана на языке SysML v2 в виде двух текстовых моделей:

– RateTaxContext.sysml – акторы (ИП, бухгалтер), система RateTaxApp, внешний сервис NbrbApi, порт RatesPort, передаваемые элементы RateRequest, RateResponse, TaxReport и пять вариантов использования UC-01…UC-05;

– RateTaxUI.sysml – библиотека элементов интерфейса (Window, Button, ComboBox, DatePicker, DataGrid, Chart и др.) и три экрана (mainScreen, incomeScreen, reportScreen) с атрибутами элементов и машиной состояний навигации.

Полные тексты моделей приведены в листингах А.3 и А.4 приложения А.

По моделям были построены графические представления и сохранены в формате PNG в каталоге lab3/srs/img. Каждый мокап оформлен как рамка диаграммы SysML v2 с указанием элемента модели, а подписи элементов управления имеют вид «имя : Тип», что обеспечивает трассируемость рисунка к модели. Диаграммы приведены в приложении Б: контекст системы и варианты использования – на рисунке Б.1, прототипы экранов «Курс на дату», «Книга доходов» и «Расчёт налога» – на рисунках Б.2–Б.4.

4.5 Заполнение шаблона спецификации требований

На основе шаблона srs/srs-template.md был создан документ lab3/srs/srs-ratetax.md на русском языке. Были заполнены разделы: введение (назначение, область применения, глоссарий, ссылки), обзор продукта (перспектива, функции F1–F6, ограничения C-1…C-5, классы пользователей, допущения A-1…A-4, распределение требований по релизам), требования (внешние интерфейсы, функциональные требования, атрибуты качества, проектирование и реализация), матрица верификации и приложения. Раздел 3.6 «ИИ/МО» помечен как неприменимый. В документ добавлены ссылки на PNG-файлы и модели SysML v2. Пример оформления требования приведён в листинге А.5 приложения А.

4.6 Фиксация документа в отдельной ветке и публикация

Подготовленные материалы были проиндексированы и зафиксированы в ветке lab3, после чего ветка была отправлена в личный форк на GitHub (листинг А.6 приложения А).

4.7 Получение замечаний бизнес-сообщества через pull request

После публикации ветки lab3 на GitHub был создан pull request из ветки lab3 в ветку main. Ссылка на pull request была передана напарнику (Заказчику), а затем преподавателю. Замечания оставлялись в виде комментариев к строкам документа. Результаты анализа замечаний приведены в таблице 4.2.

Таблица 4.2 – Анализ замечаний к pull request

№	Замечание	Решение	Обоснование
1	Не описано, какой курс брать, если оплата пришла в выходной день	Принято	Добавлено требование REQ-FUNC-011
2	Потеря базы данных означает потерю книги доходов	Принято	Добавлено требование REQ-REL-002 (резервное копирование)
3	Бухгалтер ведёт нескольких ИП — нужны профили	Принято частично	REQ-FUNC-012 с приоритетом «может», перенесено на релиз 2.0 из-за сроков
4	Добавить автоматическую подачу декларации в налоговую	Отклонено	Нет открытого API, требуется ЭЦП и юридическая ответственность; вне границ продукта
5	Сделать мобильную версию	Отклонено	Противоречит ограничениям C-1, C-3, C-5; основной сценарий — работа с документами на ПК
6	Требование «приложение должно работать быстро» непроверяемо	Принято	REQ-PERF-001 переформулировано с численными метриками, добавлено REQ-PERF-002

Замечания 4 и 5 были отклонены как неконструктивные в рамках текущего проекта: они расширяют границы продукта, противоречат согласованным ограничениям и не могут быть выполнены в сроки семестра. Отклонение было аргументировано Заказчику в ответных комментариях к pull request.

4.8 Внесение корректировок

По результатам анализа в документ были внесены изменения: добавлены требования REQ-FUNC-011 (курс на выходные и праздничные дни), REQ-FUNC-012 (профили ИП, релиз 2.0), REQ-REL-002 (резервное копирование), REQ-PERF-002 (объём данных), переформулировано REQ-PERF-001, обновлены таблица распределения требований, матрица верификации и история изменений; версия документа повышена до 0.2. В приложение 5.3 SRS добавлена таблица анализа замечаний. Фрагмент обновлённой спецификации приведён в листинге А.7, команды фиксации версии 0.2 – в листинге А.8 приложения А.

Слияние ветки lab3 в ветку main выполняется после окончательного согласования документа с преподавателем.

4.9 Смена ролей

Во второй части работы роли были изменены: автор выступил Менеджером проекта для напарника – провёл интервью по его проекту, а затем, в роли представителя бизнес-сообщества, оставил замечания к его pull request. При этом особое внимание уделялось проверяемости формулировок, наличию критериев приёмки и чётким границам продукта.

5 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

5.1 Какие ресурсы ограничены и волнуют заказчика?

Заказчика прежде всего волнуют три взаимосвязанных ресурса, образующих «железный треугольник» управления проектом: деньги (бюджет разработки и стоимость владения), время (срок вывода продукта на рынок, поскольку запоздавший продукт может потерять ценность) и объём/качество (набор функций и их надёжность). Изменение одного из них неизбежно влияет на остальные. Помимо этого ограничены человеческие ресурсы (собственное время Заказчика и его сотрудников на участие в проекте) и репутационные риски.

5.2 Хорошо ли, когда заказчик видит разработанное ПО только один раз, когда оно готово?

Нет. Если Заказчик увидит продукт только в конце, велик риск, что результат не совпадёт с его ожиданиями: требования могли быть поняты неверно, а бизнес-условия – измениться. Исправление ошибок на этапе приёмки обходится в десятки раз дороже, чем на этапе требований. Регулярная демонстрация промежуточных результатов (прототипов, инкрементов) позволяет вовремя получить обратную связь и скорректировать направление разработки.

5.3 Стоит ли обращаться к заказчику, если какие-либо требования не ясны?

Да, стоит. Уточнение неясных требований у Заказчика устраняет неоднозначность, исключает ошибочные предположения разработчика и снижает объём последующих переделок. Кроме того, постоянное вовлечение повышает доверие Заказчика и его ответственность за результат. В данной работе, например, уточнение вопроса о курсе на выходные дни привело к появлению отдельного требования REQ-FUNC-011.

5.4 Что такое требование к ПО?

Требование к ПО – это задокументированное условие или возможность, которой должна обладать система или её компонент, чтобы удовлетворить потребность пользователя, договор, стандарт или иной формальный документ. Требования бывают бизнес-, пользовательскими, функциональными и нефункциональными; кроме того, выделяют ограничения. Хорошее требование однозначно, проверяемо, необходимо, осуществимо и трассируемо.

5.5 Обладает ли заказчик в начале разработки ясным видением результата? Как уменьшить его затраты?

Как правило, нет: Заказчик хорошо понимает свою проблему, но смутно представляет конечное решение. Для уменьшения затрат следует: проводить тщательное выявление требований (интервью, анализ процессов); фиксировать бизнес-цели и границы проекта; использовать прототипы и мокапы для раннего согласования интерфейса; разрабатывать продукт итеративно, начиная с минимально жизнеспособной версии (MVP); приоритизировать требования и откладывать второстепенные на последующие релизы.

5.6 Меняются ли требования во время разработки? Как свести затраты на переделки к минимуму?

Да, требования меняются практически всегда: изменяется рынок, законодательство, понимание задачи самим Заказчиком. Чтобы минимизировать затраты, необходимо: применять итеративные (гибкие) методологии с короткими циклами и регулярной демонстрацией; вести требования под версионным контролем с историей изменений и формальной процедурой управления изменениями (в данной работе – pull request и ревью); обеспечивать трассируемость требований к коду и тестам для быстрой оценки влияния изменения; проектировать архитектуру модульной, изолируя изменчивые части (например, адаптер внешнего API); согласовывать каждое изменение с оценкой его стоимости и сроков.

ВЫВОДЫ

В ходе выполнения лабораторной работы № 3 были получены следующие результаты:

– определён проект на весь курс – desktop-приложение «КурсНалог» для пересчёта валютной выручки ИП по официальным курсам НБ РБ и расчёта налога, определены стек технологий (C++17, классический WinAPI, GDI, WinINet), внешний сервис и границы продукта;

– освоено выявление требований: в ролевой игре с напарником проведено интервью, сформулированы три бизнес-цели, классы пользователей, ограничения и допущения;

– разработана графическая часть на SysML v2: созданы текстовые модели контекста, вариантов использования и прототипа интерфейса, по ним построены четыре диаграммы в формате PNG (приложение Б);

– задокументированы требования: заполнен шаблон SRS – 12 функциональных требований и 8 требований к внешним интерфейсам и качеству обслуживания с идентификаторами, критериями приёмки и методами верификации;

– отработана совместная работа с требованиями: документ зафиксирован в отдельной ветке, проведено ревью через pull request, из шести замечаний четыре приняты (одно частично), два обоснованно отклонены, выпущена версия 0.2, репозиторий синхронизирован с удалённым;

– материалы опубликованы: спецификация, модели SysML v2 и отчёт размещены в папке lab3 ветки lab3 репозитория akutkola-source/SDLC на GitHub.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинги моделей SysML v2, фрагментов спецификации и команд Git

Листинг А.1 – Синхронизация репозитория и создание ветки lab3

```
git checkout main
git pull --ff-only origin main
Updating 1cb2401..7f16a00
Fast-forward
 lab1/src/Source.cpp      | 382 ++++++++++++++++++
 lab1/src/age runing.slnx |   7 +
git checkout -b lab3
Switched to a new branch 'lab3'
mkdir lab3\srs\sysml lab3\srs\img lab3\docs
```

Листинг А.2 – Структура папки lab3 в репозитории

```
lab3/
├── README.md                  — описание работы
├── srs/
│   ├── srs-ratetax.md         — спецификация требований (v0.2)
│   ├── sysml/
│   │   ├── RateTaxContext.sysml — контекст, варианты использования
│   │   └── RateTaxUI.sysml      — прототип интерфейса
│   └── img/                   — context.png, ui-main.png,
│                                ui-income.png, ui-report.png
└── docs/
    └── Otchet_Lab3_SRS_Shurhay_434701.docx — данный отчёт
```

Листинг А.3 – Модель контекста системы RateTaxContext.sysml

```
// Контекстная модель и варианты использования системы «КурсНалог»
// Язык: SysML v2 (текстовая нотация)

package RateTaxContext {
    private import ScalarValues::*;

    // ---------- Акторы ----------
    part def IndividualEntrepreneur {
        doc /* Индивидуальный предприниматель, получающий выручку в
            иностранной валюте */
    }
    part def Accountant {
        doc /* Бухгалтер на аутсорсе, ведущий учёт нескольких ИП */
    }
    part def NbrbApi {
        doc /* Внешний сервис: REST API Национального банка Республики
            Беларусь (api.nbrb.by) */
        attribute baseUrl : String = "https://api.nbrb.by/exrates";
    }

    // ---------- Система ----------
    part def RateTaxApp {
        doc /* Desktop-приложение «КурсНалог» */
        port nbrbPort : RatesPort;
        port fileExport : ExportPort;
    }

    port def RatesPort {
```

```
        out item rateRequest : RateRequest;
        in item rateResponse : RateResponse;
    }
    port def ExportPort {
        out item report : TaxReport;
    }

    item def RateRequest {
        attribute curAbbreviation : String;
        attribute onDate : String;
    }
    item def RateResponse {
        attribute curScale : Integer;
        attribute curOfficialRate : Real;
    }
    item def TaxReport {
        attribute periodFrom : String;
        attribute periodTo : String;
        attribute totalIncomeByn : Real;
        attribute taxAmountByn : Real;
    }

    // ---------- Контекст ----------
    part context {
        part user : IndividualEntrepreneur;
        part accountant : Accountant;
        part app : RateTaxApp;
        part nbrb : NbrbApi;

        connect app.nbrbPort to nbrb;
    }

    // ---------- Варианты использования ----------
    use case def ViewRateOnDate {
        doc /* UC-01: получить официальный курс выбранной валюты на дату
            */
        subject app : RateTaxApp;
        actor user : IndividualEntrepreneur;
    }
    use case def RegisterIncome {
        doc /* UC-02: зарегистрировать поступление выручки в валюте и
            пересчитать в BYN */
        subject app : RateTaxApp;
        actor user : IndividualEntrepreneur;
        include use case getRate : ViewRateOnDate;
    }
    use case def CalculateTax {
        doc /* UC-03: рассчитать налоговую базу и сумму налога за период
            */
        subject app : RateTaxApp;
        actor user : IndividualEntrepreneur;
    }
    use case def ExportReport {
        doc /* UC-04: выгрузить книгу учёта доходов в CSV и распечатать
            расчёт (в т.ч. в PDF) */
        subject app : RateTaxApp;
        actor accountant : Accountant;
    }
    use case def AnalyseDynamics {
        doc /* UC-05: просмотреть график динамики курса за период */
        subject app : RateTaxApp;
        actor user : IndividualEntrepreneur;
```

```
    }
}
```

Листинг А.4 – Модель прототипа интерфейса RateTaxUI.sysml

```
// Прототип графического интерфейса системы «КурсНалог»
// Язык: SysML v2 (текстовая нотация). Каждый экран описан как part,
// элементы управления — как вложенные part с атрибутами расположения.
// Графическое представление: ../img/ui-main.png, ../img/ui-income.png,
// ../img/ui-report.png

package RateTaxUI {
    private import ScalarValues::*;

    // ---------- Библиотека элементов интерфейса ----------
    abstract part def Widget {
        attribute x : Integer;
        attribute y : Integer;
        attribute width : Integer;
        attribute height : Integer;
    }
    part def Window :> Widget { attribute title : String; }
    part def Label :> Widget { attribute text : String; }
    part def Button :> Widget {
        attribute caption : String;
        attribute isPrimary : Boolean = false;
    }
    part def ComboBox :> Widget {
        attribute items : String[*];
        attribute selected : String;
    }
    part def DatePicker :> Widget { attribute value : String; }
    part def NumberBox :> Widget { attribute value : Real; }
    part def DataGrid :> Widget { attribute columns : String[*]; }
    part def Chart :> Widget { attribute seriesName : String; }
    part def TabBar :> Widget {
        attribute tabs : String[*];
        attribute active : String;
    }
    part def StatusBar :> Widget { attribute text : String; }

    // ---------- Экран 1: Курс на дату (UC-01, UC-05) ----------
    part mainScreen : Window {
        attribute redefines title = "КурсНалог — Курс на дату";
        attribute redefines width = 1100;
        attribute redefines height = 680;

        part tabs : TabBar {
            attribute redefines tabs = ("Курс на дату", "Книга доходов",
                                        "Расчёт налога", "Настройки");
            attribute redefines active = "Курс на дату";
        }
        part currency : ComboBox {
            attribute redefines items = ("USD", "EUR", "RUB", "CNY",
                                         "PLN");
            attribute redefines selected = "USD";
        }
        part onDate : DatePicker;
        part btnGetRate : Button {
            attribute redefines caption = "Получить курс";
            attribute redefines isPrimary = true;
        }
        part rateCard : Label {
```

```
        attribute redefines text = "1 USD = 2,9547 BYN";
    }
    part periodFrom : DatePicker;
    part periodTo : DatePicker;
    part dynamics : Chart {
        attribute redefines seriesName = "Официальный курс НБ РБ";
    }
    part converterAmount : NumberBox;
    part converterResult : Label;
    part status : StatusBar {
        attribute redefines text = "Источник: api.nbrb.by · кэш
            обновлён";
    }
}

// ---------- Экран 2: Книга учёта доходов (UC-02) ----------
part incomeScreen : Window {
    attribute redefines title = "КурсНалог — Книга доходов";
    part btnAdd : Button {
        attribute redefines caption = "+ Добавить поступление";
        attribute redefines isPrimary = true;
    }
    part btnImport : Button {
        attribute redefines caption = "Импорт выписки (CSV)";
    }
    part grid : DataGrid {
        attribute redefines columns = ("Дата", "Документ", "Валюта",
                                       "Сумма", "Курс НБ РБ",
                                       "Сумма, BYN", "Комментарий");
    }
    part totalLabel : Label {
        attribute redefines text = "Итого за период, BYN";
    }

    part addDialog : Window {
        attribute redefines title = "Новое поступление";
        part date : DatePicker;
        part cur : ComboBox;
        part amount : NumberBox;
        part doc : Label {
            attribute redefines text = "Номер платёжного документа";
        }
        part rateHint : Label {
            attribute redefines text = "Курс подставляется
                автоматически";
        }
        part btnSave : Button {
            attribute redefines caption = "Сохранить";
            attribute redefines isPrimary = true;
        }
        part btnCancel : Button {
            attribute redefines caption = "Отмена";
        }
    }
}

// ---------- Экран 3: Расчёт налога и отчёт (UC-03,
                                                UC-04) ----------
part reportScreen : Window {
    attribute redefines title = "КурсНалог — Расчёт налога";
    part period : ComboBox {
        attribute redefines items = ("I квартал", "II квартал",
                                     "III квартал", "IV квартал",
                                     "Год");
```

```
        }
        part taxRate : NumberBox;
        part summary : DataGrid {
            attribute redefines columns = ("Показатель", "Значение,
                                           BYN");
        }
        part btnExportCsv : Button {
            attribute redefines caption = "Экспорт CSV";
        }
        part btnPrint : Button {
            attribute redefines caption = "Печать / PDF";
            attribute redefines isPrimary = true;
        }
    }

    // ---------- Навигация между экранами ----------
    state def Navigation {
        entry; then RateView;
        state RateView;
        state IncomeView;
        state ReportView;
        transition RateView_to_Income first RateView then IncomeView;
        transition Income_to_Report first IncomeView then ReportView;
        transition Report_to_Rate first ReportView then RateView;
    }
}
```

Листинг А.5 – Пример функционального требования в SRS

```
- **ID:** REQ-FUNC-003
- **Title:** Автоматический пересчёт в BYN
- **Statement:** Система должна при сохранении поступления
  подставлять курс НБ РБ на дату поступления и вычислять
  сумму в BYN = сумма × курс / Cur_Scale с округлением до копеек.
- **Rationale:** БЦ-2.
- **Acceptance Criteria:** 60 000 RUB при курсе 3,6420 за 100 RUB
  → 2 185,20 BYN.
- **Verification Method:** Test
```

Листинг А.6 – Фиксация версии 0.1 спецификации и публикация ветки lab3

```
git add lab3/srs
git commit -m "docs(lab3): add RateTax business requirements v0.1
  with SysML v2 models and UI mockups"
[lab3 2629cd3] docs(lab3): add RateTax business requirements v0.1 ...
 7 files changed
 create mode 100644 lab3/srs/img/context.png
 create mode 100644 lab3/srs/img/ui-income.png
 create mode 100644 lab3/srs/img/ui-main.png
 create mode 100644 lab3/srs/img/ui-report.png
 create mode 100644 lab3/srs/srs-ratetax.md
 create mode 100644 lab3/srs/sysml/RateTaxContext.sysml
 create mode 100644 lab3/srs/sysml/RateTaxUI.sysml
git push -u origin lab3
remote: Create a pull request for 'lab3' on GitHub by visiting:
remote:      https://github.com/akutkola-source/SDLC/pull/new/lab3
To github.com:akutkola-source/SDLC.git
```

```
 * [new branch]      lab3 -> lab3
branch 'lab3' set up to track 'origin/lab3'.
```

Листинг А.7 – Требования, добавленные и изменённые в SRS версии 0.2

```
- **ID:** REQ-FUNC-011
- **Title:** Курс на выходные и праздничные дни
- **Statement:** Если поступление приходится на день, на который
  НБ РБ не устанавливал новый курс, система должна использовать
  курс, действующий на эту дату, и помечать запись значком
  «курс на {дата установления}».
- **Verification Method:** Test

- **ID:** REQ-PERF-001
- **Statement:** Отображение курса из кэша — не более 0,5 с,
  с запросом к API — не более 3 с (95-й перцентиль);
  холодный запуск приложения — не более 3 с.
- **Verification Method:** Test
```

Листинг А.8 – Фиксация версии 0.2 после обсуждения

```
git add lab3/srs
git commit -m "docs(lab3): apply review feedback,
                    bump RateTax SRS to v0.2"
[lab3 9be5d22] docs(lab3): apply review feedback,
                    bump RateTax SRS to v0.2
```

ПРИЛОЖЕНИЕ Б
(обязательное)
Графическая часть спецификации требований на языке SysML v2

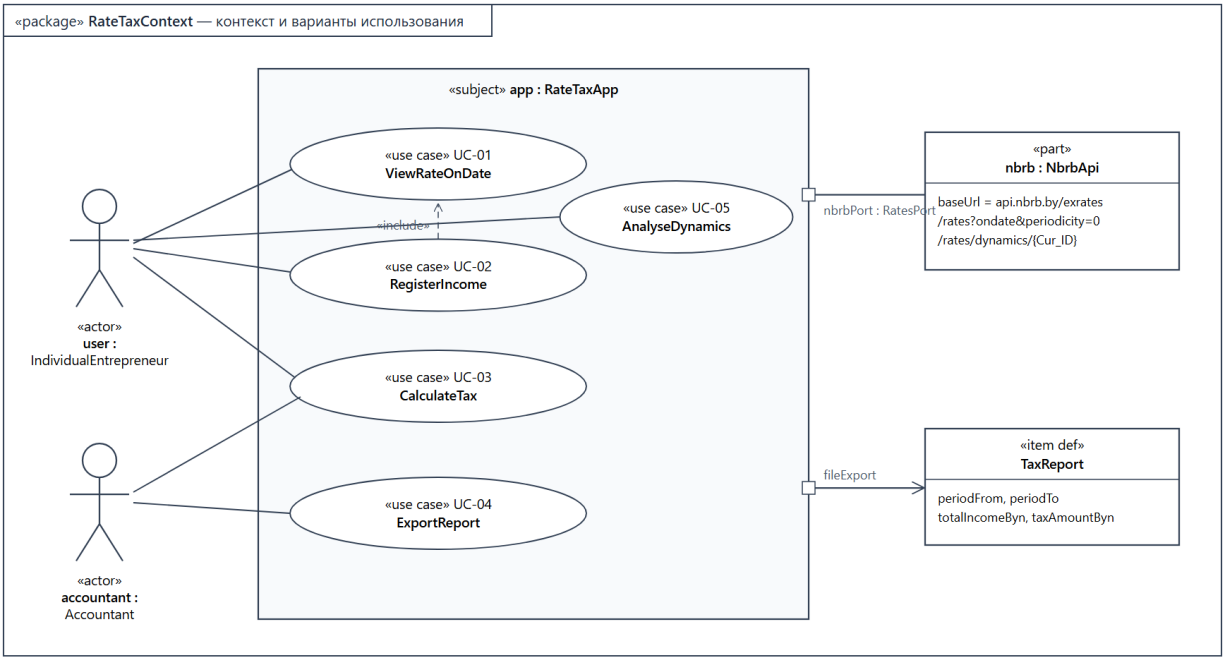


Рисунок построен по модели RateTaxContext.sysml: акторы (part def), use case def, порт RatesPort и внешний сервис NbrbApi.

Рисунок Б.1 – Контекст системы и варианты использования (RateTaxContext)

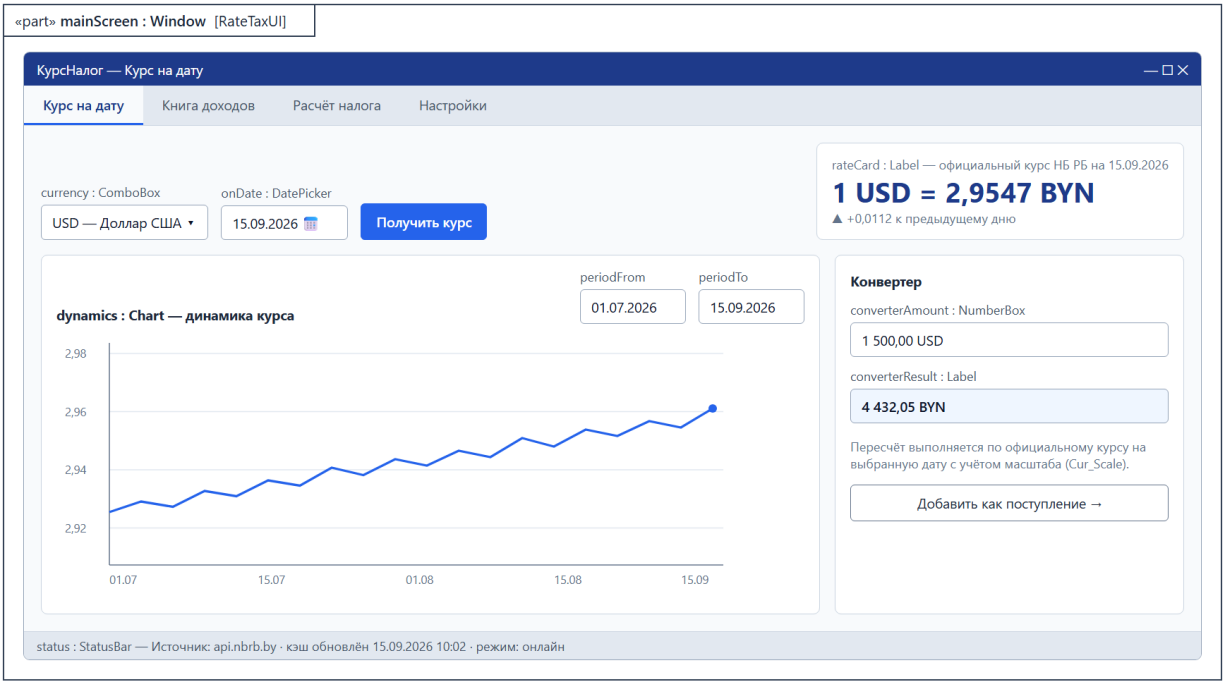


Рисунок построен по модели RateTaxUI.sysml (part mainScreen). Подписи вида «имя : Тип» соответствуют частям модели.

Рисунок Б.2 – Прототип экрана «Курс на дату» (part mainScreen)

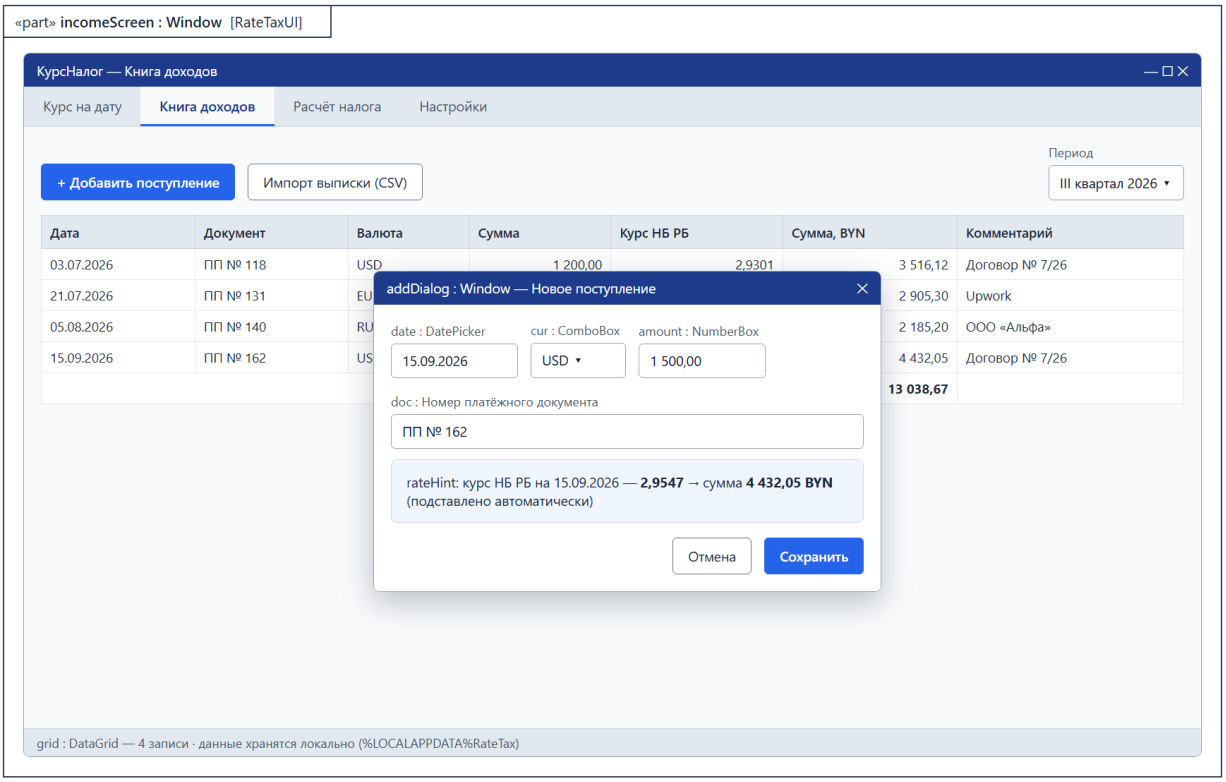


Рисунок построен по модели RateTaxUI.sysml (part incomeScreen и вложенный part addDialog).

Рисунок Б.3 – Прототип экрана «Книга доходов» с диалогом добавления поступления (part incomeScreen)

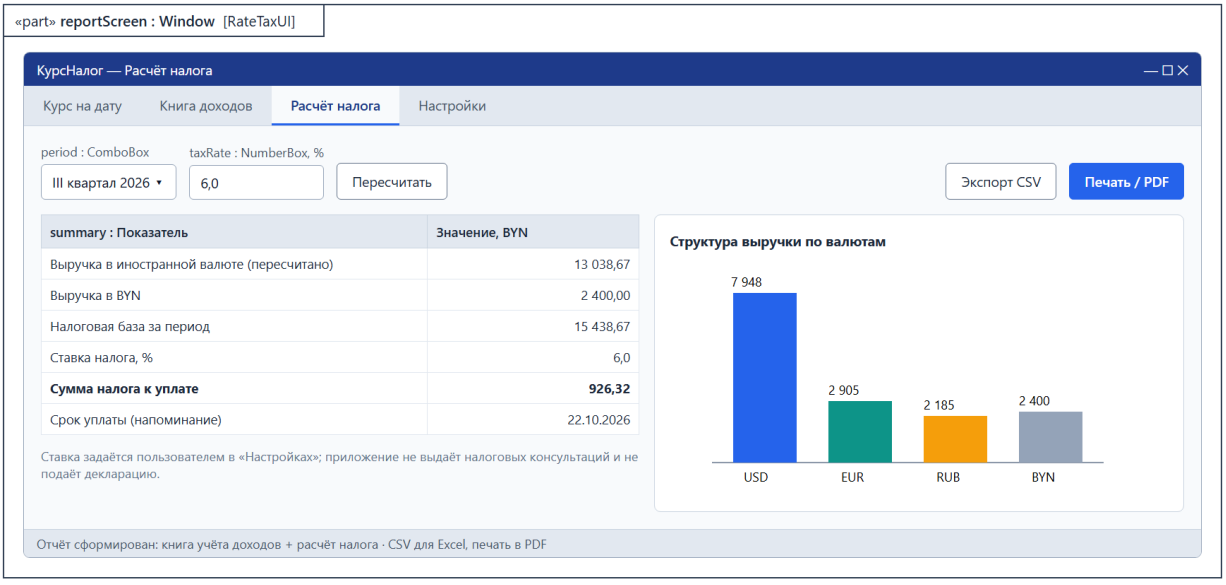


Рисунок построен по модели RateTaxUI.sysml (part reportScreen).

Рисунок Б.4 – Прототип экрана «Расчёт налога» (part reportScreen)